When I looked at Active Record for the first time, I'd been a Java programmer for years. Active Record's source code left me shocked. No sane Java coder would ever write a library that consists almost exclusively of a single huge class with many hundreds of methods. Such a library would be madness—impossible to understand and maintain!

And yet, that's exactly what Active Record's design is like: most methods in the library ultimately get rolled inside one class. But wait, it gets worse. As we'll discuss later, some of the modules that comprise Active Record don't think twice about using metaprogramming to define even more methods on their includer. To add insult to injury, even additional libraries that work with Active Record often take the liberty of extending `ActiveRecord::Base` with modules and methods of their own. You might think that the result of this relentless piling up of methods would be a tangled mass of spaghetti. But it isn't.

Consider the evidence: not only does Active Record get away with that design, it also proves easy to read and change. Many users modify and *Monkeypatch (16)* Active Record for their own purposes, and hundreds of contributors have worked on the original source code. Still, the source code evolves so quickly that the poor authors of books such as this one need to rewrite most of their content with every new edition. Active Record manages to stay stable and reliable even as it changes, and most coders are happy using the latest version of the library in their production systems.

Here is the most important guideline I learned from Active Record's design: *design techniques are relative*, and they depend on the language you're using. In Ruby, you use idioms that are different from those of other languages you might be used to. It's not that the good design rules of old suddenly grew obsolete. On the contrary, the basic tenets of design (decoupling, simplicity, no duplication) hold true in Ruby as much as they do in any other language. In Ruby, though, the techniques you wield to achieve those design goals can be surprisingly different.

Look at `ActiveRecord::Base` again. It's a huge class, but this complex class doesn't exist in the source code. Instead, it is composed at runtime by assembling loosely coupled, easy-to-test, easy-to-reuse modules. If you only need the validation features, you can include `ActiveModel::Validations` in your own class and happily ignore `ActiveRecord::Base` and all the other modules, as in the following code: